```
1544311232))
            ("Internal time" (format-time-string format '(23564
20962 864324 108000))))))
```

The output appears as an s-expression:

```
(("Form" "x faster than next" "Total runtime" "# of GCs" "Total
GC runtime")
hline

("Internal time" "1.11" "2.626460" 13 "0.838733")
("Unix timestamp" "slowest" "2.921408" 13 "0.920814"))
```

The abbreviation 'GC' refers to garbage collection. A tabular representation of the above results is given below:

"Form"	"x faster than next"	"Total runt-ime"	"# of GCs"	"Total GC runtime"
"Internal time"	"1.11"	"2.626460"	13	"0.838733"
"Unix timestamp"	"slowest"	"2.921408"	13	"0.920814"

We observe that formatting the internal time is slightly faster.

Getting the current time

The functions to obtain the current time can be compared using the following test:

```
(bench-multi :times 100000
  :forms (("Unix timestamp" (float-time))
          ("Internal time" (current-time))))
```

The results are shown below:

"Form"	"x faster than next"	"Total runt-ime"	"# of GCs"	"Total GC runtime"
"Unix times-tamp"	"10.97"	"0.018114"	0	0
"Internal time"	"slowest"	"0.198637"	2	"0.179333"

We observe that using the UNIX timestamp is faster.

Parsing

The third benchmarking exercise is to compare parsing functions on a date timestamp string. The corresponding test code is given below:

```
(let* ((s "Wed 10 Jul 2019"))
  (bench-multi :times 100000
    :forms (("ts-parse" (ts-parse s))
            ("ts-parse ts-unix" (ts-unix (ts-parse s))))))
```

The *ts-parse* function is slightly faster than the *ts-parse ts-unix* function, as seen in the results:

"Form"	"x faster than next"	"Total runt-ime"	"# of GCs"	"Total GC runtime"
"ts-parse"	"1.03"	"8.113778"	43	"2.954245"
"ts-parse ts-unix"	"slow-est"	"8.343840"	43	"3.023095"

A new timestamp versus blanking fields

The last performance comparison is between creating a new timestamp and blanking the fields. The relevant test code is as follows:

```
(let* ((a (ts-now)))
  (bench-multi :times 100000
    :ensure-equal t
    :forms (("New" (let ((ts (copy-ts a)))
                     (setq ts (ts-fill ts))
                     (make-ts :unix (ts-unix ts))))
            ("Blanking" (let ((ts (copy-ts a)))
                          (setq ts (ts-fill ts))
                          (ts-reset ts))))))
```

The output of the benchmarking exercise is given below:

"Form"	"x faster than next"	"Total runt-ime"	"# of GCs"	"Total GC runtime"
"New"	"1.00"	"16.465727"	134	"10.111131"
"Blank-ing"	"slowest"	"16.479456"	134	"10.135710"

We observe that creating a new timestamp is slightly faster than blanking the fields.

You are encouraged to read the ts.el README and *notes.org* from the GitHub repository *https://github.com/alphapapa/ts.el* for more information. END

By: Shakthi Kannan

The author is a free software enthusiast.